LLM in Action: Vibe Coding Example

- **Vibe coding:** describe what you want in plain English; the LLM generates the code directly, refined through conversation.

Example:

- Prompt: *"Check if a WhatsApp message confirms payment and extract the amount."*
  - `if "paid" in msg: return re.search(r"\$(\d+\.\d{2})", msg)`
- → Code appears instantly; developer refines with: *"also handle SGD and USD."*
- Great for prototypes — but the **LLM decides the structure**, not a spec.

What is Spec-Driven Development (SDD)?

- **SDD:** writing a clear, structured spec — requirements, data models, API contracts, edge cases — before any code is generated.
- The spec becomes the source of truth; the LLM generates code to satisfy it, not the other way around.
- Used in agentic coding tools (Claude Code, GitHub Spec Kit) via structured docs: **requirements.md, design.md, tasks.md**.
- Reviewed and version-controlled like code — enabling traceability and safe iteration.

Example: instead of *"build a payment parser,"* the spec fixes the formats and rules first — then the LLM codes to it.

Why is Spec-Driven Development Needed?

- Vibe coding is fast for prototypes but breaks down as systems grow:
- **Hallucinated logic:** LLM invents behaviour that was never intended, especially in edge cases.
- **Inconsistent output:** the same prompt on different days can generate different code.
- **Hard to review:** no shared reference to check “is this correct?” besides the code itself.
- **Hard to maintain:** teammates can't tell why a design choice was made.

Why SDD helps: it gives the LLM — and the team — an explicit, testable contract to generate, verify, and regenerate code against.

Vibe Coding vs. Spec-Driven Development

Vibe Coding

- Fast, exploratory — natural-language prompts drive the code directly.
- Best for prototypes, demos, and one-off scripts.
- Risk: hallucination, inconsistency, hard to scale or review.

Spec-Driven Development

- Deliberate, specification-first — the spec defines requirements before code generation.
- Best for production systems, team projects, and compliance-critical apps.
- Benefit: traceability, consistency, testability.

Key takeaway: vibe coding explores the idea; SDD engineers the product for scale.